Reportes Detallados de Pruebas Software de NLPolitik

En este documento se presentan los reportes detallados de todas las pruebas realizadas al sistema NLPolitik, incluyendo pruebas unitarias y de integración.

Reportes de Pruebas Unitarias

Reporte de Pruebas Unitarias: Procesador NLP

Prueba	UT-NLP-001
Título	Validar inicialización del procesador
Precondiciones	Ninguna
Cobertura	Constructor __init__() - modelo spaCy cargado
Dependencias	spaCy español
Pasos	1. Crear instancia de ProcesadorNLP
Configuración	Modelo spaCy por defecto para español
Esperado	Modelo spaCy cargado correctamente
Resultado	Modelo spaCy cargado
Estado	**APROBADO**

Tabla C.1: Prueba unitaria NLP 1 - Inicialización del procesador

Prueba	UT-NLP-002
Título	Validar limpieza de texto válido
Precondiciones	Procesador NLP inicializado
Cobertura	Método limpiar_texto() - entrada normal
Dependencias	Ninguna
Pasos	1. Proporcionar texto con URLs y caracteres especiales
Configuración	Texto: 'Hola mundo! #saludo @usuario de Nlpolitik$ %&'
Esperado	'hola mundo saludo usuario de nlpolitik'
Resultado	'hola mundo saludo usuario de nlpolitik'
Estado	**APROBADO**

Tabla C.2: Prueba unitaria NLP 2 - Limpieza de texto válido

Prueba	UT-NLP-003
Título	Validar limpieza de texto
Precondiciones	Procesador NLP inicializado
Cobertura	Método limpiar_texto() - entrada
Dependencias	Ninguna
Pasos	1. Proporcionar texto en formato UTF-8
Configuración	Entrada (str): 'Texto con acentos: áéíóú y eñe'
Esperado	'texto con acentos: áéíóú y eñe'
Resultado	'texto con acentos: áéíóú y eñe'
Estado	**APROBADO**

Tabla C.3: Prueba unitaria NLP 3 - Limpieza de texto UTF-8

Prueba	UT-NLP-004
Título	Validar limpieza de texto vacío
Precondiciones	Procesador NLP inicializado
Cobertura	Método limpiar_texto() - entrada vacía/nula
Dependencias	Ninguna
Pasos	1. Proporcionar string vacío y None
Configuración	Entrada: string vacío ('None')
Esperado	Manejo sin errores
Resultado	Caso '': resultado '' - APROBADO Caso 'None': error 'NoneType' object has no attribute 'lower' - FALLIDO
Estado	**FALLIDO**

Tabla C.4: Prueba unitaria NLP 4 - Limpieza de texto vacío

Prueba	UT-NLP-005
Título	Validar extracción de frases con texto válido
Precondiciones	Procesador NLP inicializado
Cobertura	Método extraer_frases_y_pos() - entrada normal
Dependencias	spaCy con modelo español
Pasos	1. Proporcionar texto con oraciones completas
Configuración	Entrada: 'El gobierno debe mejorar la economía,...'
Esperado	Lista de frases extraídas
Resultado	1 frases extraídas Frase 1: el gobierno debe mejorar la economía,...
Estado	**APROBADO**

Tabla C.5: Prueba unitaria NLP 5 - Extracción de frases válidas

Prueba	UT-NLP-006
Título	Validar extracción con texto muy corto
Precondiciones	Procesador NLP inicializado
Cobertura	Método extraer_frases_y_pos() - entrada corta
Dependencias	spaCy con modelo español
Pasos	1. Proporcionar texto de 2 palabras
Configuración	Entrada: 'Hola mundo'
Esperado	0 frases (no cumple longitud)
Resultado	0 frases extraídas
Estado	**APROBADO**

Tabla C.6: Prueba unitaria NLP 6 - Extracción con texto corto

Prueba	UT-NLP-007
Título	Validar procesamiento completo de archivo
Precondiciones	Procesador NLP inicializado
Cobertura	Método procesar_archivo_individual()
Dependencias	spaCy con modelo español
Pasos	1. Ejecutar procesar_archivo_individual()
Configuración	Artículo de prueba: Título: Artículo de prueba, Contenido: 'La economía del país necesita reformas profundas. ...'
Esperado	Diccionario con 7 campos
Resultado	Estructura con campos: ['titulo', 'texto_limpio', 'caracteres_eliminados', 'frases', 'etiquetas_pos', 'longitudes_frases', 'total_frases']
Estado	**APROBADO**

Tabla C.7: Prueba unitaria NLP 7 - Procesamiento completo de archivo

Prueba	UT-NLP-008
Título	Validar cálculo de métricas globales
Precondiciones	Procesador NLP inicializado
Cobertura	Método calcular_metricas_globales() - agregación correcta
Dependencias	Resultados simulados de procesamiento
Pasos	1. Crear lista de resultados simulados
Configuración	Resultados simulados: 3 archivos
Esperado	total_caracteres=30, total_frases=9, promedio=9.33
Resultado	total_caracteres=30, total_frases=9, promedio=9.33
Estado	**APROBADO**

Tabla C.8: Prueba unitaria NLP 8 - Cálculo de métricas globales

Prueba	UT-NLP-009
Título	Validar consistencia en transformaciones
Precondiciones	Procesador NLP inicializado
Cobertura	Métodos múltiples - misma entrada produce misma salida
Dependencias	Ninguna
Pasos	1. Ejecutar limpieza 20 veces con misma entrada
Configuración	Entrada: 'Texto con espacios extras!! @mencion #hashtag'
Esperado	20 resultados idénticos
Resultado	20 resultados únicos
Estado	**APROBADO**

Tabla C.9: Prueba unitaria NLP 9 - Consistencia en transformaciones

Prueba	UT-NLP-010
Título	Validar manejo de excepciones en extracción
Precondiciones	Procesador NLP inicializado
Cobertura	Método extraer_frases_y_pos() - entrada problemática
Dependencias	spaCy con modelo español
Pasos	1. Proporcionar texto con caracteres extraños
Configuración	Entrada: Texto muy largo (10006 caracteres)
Esperado	Método no lanza excepción, retorna listas
Resultado	0 frases, 0 etiquetas POS
Estado	**APROBADO**

Tabla C.10: Prueba unitaria NLP 10 - Manejo de excepciones en extracción

Resumen de ejecución: Ran 10 tests in 19.216s
Estadísticas finales:

- **Total de pruebas:** 10
- **Pruebas aprobadas:** 9 (90 %)
- **Pruebas fallidas:** 1 (10 %) - UT-NLP-004
- **Tiempo total de ejecución:** 19.216 segundos

Reporte de Pruebas Unitarias: Procesador Embeddings

Prueba	UT-EMB-001
Título	Validar inicialización del procesador
Precondiciones	Ninguna
Cobertura	Constructor __init__() - modelo cargado
Dependencias	SentenceTransformer español
Pasos	1. Crear instancia de ProcesadorEmbedding
Configuración	Modelo: hiiamsid/sentence_similarity_spanish_es
Esperado	Modelo SentenceTransformer cargado
Resultado	Modelo cargado, dimensión: 768D Cargando modelo SentenceTransformer (modo BETO simulado): hiiamsid/sentence_similarity_spanish_es
Estado	**APROBADO**

Tabla C.11: Prueba unitaria Embeddings 1 - Inicialización

Prueba	UT-EMB-002
Título	Validar embedding de frase válida
Precondiciones	Procesador inicializado
Cobertura	Método embed_palabra() - entrada normal
Dependencias	Modelo SentenceTransformer
Pasos	1. Generar embedding para frase simple
Configuración	Entrada: 'el gobierno debe mejorar la economía' Modelo: hiiamsid/sentence_similarity_spanish_es
Esperado	Vector numpy de 768 dimensiones
Resultado	Vector shape: (768,), tipo: <class 'numpy.ndarray'> Valores muestra: [-1.3981036 -0.02856577 0.5108489] Cargando modelo SentenceTransformer (modo BETO simulado): hiiamsid/sentence_similarity_spanish_es
Estado	**APROBADO**

Tabla C.12: Prueba unitaria Embeddings 2 - Embedding de frase válida

Prueba	UT-EMB-003
Título	Validar embedding de frase vacía
Precondiciones	Procesador inicializado
Cobertura	Método embed_palabra() - entrada vacía
Dependencias	Modelo SentenceTransformer
Pasos	1. Generar embedding para string vacío
Configuración	Entrada: (string vacío),
Esperado	Vector de ceros de 768D
Resultado	Vector shape: (768,), suma: 0.00
Estado	**APROBADO**

Tabla C.13: Prueba unitaria Embeddings 3 - Embedding de frase vacía

Prueba	UT-EMB-004
Título	Validar embedding con contexto
Precondiciones	Procesador inicializado
Cobertura	Método embed_frase_con_contexto()
Dependencias	Modelo SentenceTransformer
Pasos	1. Generar embedding con documentos de origen
Configuración	Entrada: Documentos simulados con una frase
Esperado	Vector concatenado de 1536D
Resultado	Vector shape: (1536,)
Estado	**APROBADO**

Tabla C.14: Prueba unitaria Embeddings 4 - Embedding con contexto

Prueba	UT-EMB-005
Título	Validar embedding sin documentos
Precondiciones	Procesador inicializado
Cobertura	Método embed_frase_con_contexto() - sin documentos
Dependencias	Modelo SentenceTransformer
Pasos	1. Generar embedding con lista vacía de documentos
Configuración	Entrada: Documentos: [] (vacío), simulado
Esperado	Vector de 1536D con mitad ceros
Resultado	Vector shape: (1536,)
Estado	**APROBADO**

Tabla C.15: Prueba unitaria Embeddings 5 - Embedding sin documentos

Prueba	UT-EMB-006
Título	Validar embedding de lista de frases
Precondiciones	Procesador inicializado
Cobertura	Método embed_lista_con_contexto()
Dependencias	Modelo SentenceTransformer
Pasos	1. Generar embeddings para diccionario de frases
Configuración	Entrada: Lista de documentos simulados
Esperado	Matriz de 3x1536 y mapeo de 3 frases
Resultado	Matriz shape: (3, 1536), mapeo keys: 3
Estado	**APROBADO**

Tabla C.16: Prueba unitaria Embeddings 6 - Embedding de lista de frases

Prueba	UT-EMB-007
Título	Validar consistencia en embeddings
Precondiciones	Procesador inicializado
Cobertura	Método embed_palabra() - consistencia con 20 ejecuciones
Dependencias	Modelo SentenceTransformer
Pasos	1. Generar embedding 20 veces para misma frase
Configuración	Entrada: Frase: 'la economía necesita crecimiento sostenible' Generando 20 embeddings...
Esperado	20 embeddings idénticos
Resultado	20 embeddings idénticos (similitud: promedio=1.000000).
Estado	**APROBADO (todos los embeddings son idénticos)**

Tabla C.17: Prueba unitaria Embeddings 7 - Consistencia en embeddings

Prueba	UT-EMB-008
Título	Validar métodos de dimensión
Precondiciones	Procesador inicializado
Cobertura	Métodos get_dimension() y get_dimension_con_contexto()
Dependencias	Ninguna
Pasos	1. Obtener dimensiones base y con contexto
Configuración	Llamada a métodos
Esperado	Dimensión contexto (1536) = 2 × dimensión base (768)
Resultado	Base: 768D, Contexto: 1536D
Estado	**APROBADO**

Tabla C.18: Prueba unitaria Embeddings 8 - Métodos de dimensión

Resumen de ejecución: Ran 8 tests in 18.394s
Estadísticas finales:

- **Total de pruebas:** 8
- **Pruebas aprobadas:** 8 (100 %)
- **Pruebas fallidas:** 0 (0 %)
- **Tiempo total de ejecución:** 18.394 segundos

Reporte de Pruebas Unitarias: Procesador Autoencoder

Prueba	UT-AE-001
Título	Validar inicialización del autoencoder
Precondiciones	Ninguna
Cobertura	Constructor __init__() - modelos creados
Dependencias	TensorFlow/Keras
Pasos	1. Crear instancia de ProcesadorAutoencoder
Configuración	Dimensión: 10D → 4D Autoencoder MEJORADO: 10D → 4D (reducción: 40.0 %)
Esperado	Autoencoder y encoder creados correctamente
Resultado	Autoencoder: SÍ, Encoder: SÍ, Dimensión: 10D → 4D
Estado	**APROBADO**

Tabla C.19: Prueba unitaria Autoencoder 1 - Inicialización

Prueba	UT-AE-002
Título	Validar estructura de los modelos
Precondiciones	Autoencoder inicializado
Cobertura	Modelos - dimensiones de entrada/salida
Dependencias	TensorFlow/Keras
Pasos	1. Verificar dimensiones de capas de entrada/salida
Configuración	* Autoencoder MEJORADO: 10D → 4D (reducción: 40.0 %)
Esperado	Autoencoder: 10→10, Encoder: 10→4
Resultado	Autoencoder: 10→10, Encoder: 10→4
Estado	**APROBADO**

Tabla C.20: Prueba unitaria Autoencoder 2 - Estructura de modelos

Prueba	UT-AE-003
Título	Validar codificación mantiene dimensiones
Precondiciones	Autoencoder inicializado
Cobertura	Método codificar() - dimensiones de salida
Dependencias	TensorFlow/Keras
Pasos	1. Codificar datos de prueba y verificar dimensiones
Configuración	Entrada shape: (5, 10)
Esperado	Shape: (5, 4)
Resultado	Shape: (5, 4), (reducción de 10D $\rightarrow$ 4D)
Estado	**APROBADO**

Tabla C.21: Prueba unitaria Autoencoder 3 - Codificación mantiene dimensiones

Prueba	UT-AE-004
Título	Validar consistencia en codificación
Precondiciones	Autoencoder inicializado
Cobertura	Método codificar() - consistencia 20 ejecuciones
Dependencias	TensorFlow/Keras
Pasos	1. Codificar mismos datos 20 veces
Configuración	Entrada shape: (3, 10) Generar 20 codificaciones iguales...
Esperado	20 codificaciones idénticas
Resultado	Todas iguales (de 10D $\rightarrow$ 4D (reducción: 40.0 %))
Estado	**APROBADO**

Tabla C.22: Prueba unitaria Autoencoder 4 - Consistencia en codificación

Prueba	UT-AE-005
Título	Validar codificación con entrada de ceros
Precondiciones	Autoencoder inicializado
Cobertura	Método codificar() - entrada de ceros
Dependencias	TensorFlow/Keras
Pasos	1. Codificar matriz de ceros
Configuración	Entrada: matriz de ceros (2, 10)
Esperado	Codificación cercana a cero (debido a capas BatchNorm)
Resultado	Shape: (2, 4), (reducción de 10D $\rightarrow$ 4D)
Estado	**APROBADO**

Tabla C.23: Prueba unitaria Autoencoder 5 - Codificación con ceros

Prueba	UT-AE-006
Título	Validar entrenamiento básico
Precondiciones	Autoencoder inicializado
Cobertura	Método entrenar() - ejecución sin errores
Dependencias	TensorFlow/Keras
Pasos	1. Entrenar con pocos epochs y datos simples
Configuración	Datos entrenamiento: (10, 10) Iniciando entrenamiento (2 epochs)...
Esperado	Entrenamiento ejecutado sin errores
Resultado	Entrenamiento completado sin excepciones (reducción de 10D → 4D)
Estado	**APROBADO**

Tabla C.24: Prueba unitaria Autoencoder 6 - Entrenamiento básico

Prueba	UT-AE-007
Título	Validar reducción dimensional efectiva
Precondiciones	Autoencoder inicializado
Cobertura	Codificación - reducción de dimensiones
Dependencias	TensorFlow/Keras
Pasos	1. Verificar que latent_dim <input_dim
Configuración	Entrada: configuración base de latent_dim e input_dim
Esperado	Reducción dimensional efectiva (60.0 %)
Resultado	Input: 10D → Latente: 4D (reducción: 60.0 %)
Estado	**APROBADO**

Tabla C.25: Prueba unitaria Autoencoder 7 - Reducción dimensional

Prueba	UT-AE-008
Título	Validar codificación con lotes variables
Precondiciones	Autoencoder inicializado
Cobertura	Método codificar() - diferentes tamaños de batch
Dependencias	TensorFlow/Keras
Pasos	1. Codificar con batch de 1, 5 y 10 muestras
Configuración	Batch 1: entrada (1, 10) → salida (1, 4) Batch 3: entrada (3, 10) → salida (3, 4) Batch 7: entrada (7, 10) → salida (7, 4)
Esperado	Todos mantienen dimensión latente: 4D
Resultado	Resultados: batch0→0×4, batch1→1×4, batch2→2×4
Estado	**APROBADO**

Tabla C.26: Prueba unitaria Autoencoder 8 - Codificación con lotes variables

Prueba	UT-AE-009
Título	Validar resumen del modelo disponible
Precondiciones	Autoencoder inicializado
Cobertura	Modelo - método summary() disponible
Dependencias	TensorFlow/Keras
Pasos	1. Verificar que se puede llamar a summary()
Configuración	Entrada: output de metodod summary
Esperado	Resumen del modelo accesible
Resultado	Método summary() disponible y funcional
Estado	**APROBADO**

Tabla C.27: Prueba unitaria Autoencoder 9 - Resumen del modelo

Prueba	UT-AE-010
Título	Validar codificación con valores extremos
Precondiciones	Autoencoder inicializado
Cobertura	Método codificar() - valores muy grandes/pequeños
Dependencias	TensorFlow/Keras
Pasos	1. Codificar con valores extremos
Configuración	Entrada: valores extremos (3, 10)
Esperado	Manejo robusto de valores extremos (sin NaN/Inf)
Resultado	Codificación exitosa, shape: (3, 4), rango: [-1.00, 1.00] NaN: NO, Inf: NO
Estado	**APROBADO**

Tabla C.28: Prueba unitaria Autoencoder 10 - Codificación con valores extremos

Resumen de ejecución: Ran 10 tests in 13.093s
Estadísticas finales:

- **Total de pruebas:** 10
- **Pruebas aprobadas:** 10 (100 %)
- **Pruebas fallidas:** 0 (0 %)
- **Tiempo total de ejecución:** 13.093 segundos

Reporte de Pruebas Unitarias: Procesador Clustering

Prueba	UT-CLUS-001
Título	Validar inicialización con KMeans
Precondiciones	Ninguna
Cobertura	Constructor __init__() - método kmeans
Dependencias	scikit-learn
Pasos	1. Crear instancia con método kmeans
Configuración	Método: kmeans, n_clusters por defecto
Esperado	Procesador inicializado con KMeans
Resultado	Método: kmeans, n_clusters: 10
Estado	**APROBADO**

Tabla C.29: Prueba unitaria Clustering 1 - Inicialización con KMeans

Prueba	UT-CLUS-002
Título	Validar KMeans encuentra clusters
Precondiciones	Datos sintéticos con 3 clusters
Cobertura	Método aplicar() - KMeans con datos claros
Dependencias	scikit-learn
Pasos	1. Estimar clusters, 2. Aplicar KMeans
Configuración	Entrada: Datos: (30, 5) (3 clusters sintéticos)
Esperado	3 clusters encontrados (estimados automáticamente)
Resultado	k estimado: 3, 3 clusters encontrados: [0 1 2]
Estado	**APROBADO**

Tabla C.30: Prueba unitaria Clustering 2 - KMeans con estimación automática

Prueba	UT-CLUS-003
Título	Validar AGNES encuentra clusters
Precondiciones	Datos sintéticos con 3 clusters
Cobertura	Método aplicar() - AGNES con datos claros
Dependencias	scikit-learn
Pasos	1. Estimar clusters, 2. Aplicar AGNES
Configuración	Entrada: Datos: (30, 5) (3 clusters sintéticos)
Esperado	3 clusters encontrados (estimados automáticamente)
Resultado	k estimado: 3, 3 clusters encontrados: [0 1 2]
Estado	**APROBADO**

Tabla C.31: Prueba unitaria Clustering 4 - AGNES con estimación automática

Prueba	UT-CLUS-004
Título	Validar DBSCAN con estimación automática de clusters
Precondiciones	Datos muy separados con 3 clusters
Cobertura	Método aplicar() - DBSCAN
Dependencias	scikit-learn, kneed
Pasos	1. Crear procesador, 2. Estimar eps, 3. Aplicar clustering
Configuración	Entrada: Datos: (30, 5) (3 clusters sintéticos)
Esperado	3 clusters encontrados (sin outliers, eps estimado automáticamente)
Resultado	3 clusters, outliers: 0, eps usado: 0.5000, min_samples: 3
Estado	**APROBADO**

Tabla C.32: Prueba unitaria Clustering 3 - DBSCAN con estimación automática

Prueba	UT-CLUS-005
Título	Validar resumen de clustering
Precondiciones	Clustering aplicado con estimación automática
Cobertura	Método resumen_clustering() - conteo por cluster
Dependencias	scikit-learn
Pasos	1. Estimar clusters, 2. Aplicar clustering, 3. Obtener resumen
Configuración	Entrada: Datos: (30, 5) (3 clusters sintéticos)
Esperado	Resumen con 3 clusters (estimados), total 30 muestras
Resultado	Clusters: 3 (k estimado: 3), Total muestras: 30
Estado	**APROBADO**

Tabla C.33: Prueba unitaria Clustering 5 - Resumen con estimación automática

Prueba	UT-CLUS-006
Título	Validar actualización parámetros DBSCAN
Precondiciones	Procesador con método DBSCAN
Cobertura	Método actualizar_parametros_dbscan()
Dependencias	scikit-learn
Pasos	1. Crear procesador DBSCAN, 2. Actualizar parámetros
Configuración	Entrada: Datos: (30, 5) (3 clusters sintéticos)
Esperado	Parámetros actualizados: eps=3.1188, min_samples=8
Resultado	Éxito en actualización. eps: 0.5000 $\rightarrow$ 3.1188, min_samples: 5 $\rightarrow$ 8
Estado	**APROBADO**

Tabla C.34: Prueba unitaria Clustering 6 - Actualización automática de parámetros DBSCAN

Prueba	UT-CLUS-007
Título	Validar DBSCAN con datos separados
Precondiciones	Datos muy separados (para DBSCAN)
Cobertura	Método aplicar() - DBSCAN con estimación automática de eps
Dependencias	scikit-learn, kneed
Pasos	1. Estimar eps, 3. Aplicar DBSCAN
Configuración	Entrada: Datos: (15, 2) (3 clusters muy separados)
Esperado	3 clusters encontrados (sin outliers, eps estimado automáticamente)
Resultado	3 agrupaciones válidas. Outliers: 0, eps usado: 0.5000
Estado	**APROBADO**

Tabla C.35: Prueba unitaria Clustering 7 - DBSCAN con estimación automática de parámetros

Prueba	UT-CLUS-008
Título	Validar métodos getter
Precondiciones	Clustering aplicado con estimación automática
Cobertura	Métodos obtener_etiquetas() y obtener_modelo()
Dependencias	scikit-learn
Pasos	1. Estimar k, 2. Aplicar clustering, 3. Obtener resultado
Configuración	Entrada: Datos: (20, 2) (2 clusters sintéticos)
Esperado	Getter devuelven mismos objetos que aplicar()
Resultado	k estimado: 2. Etiquetas iguales: True, Modelo igual: True
Estado	**APROBADO**

Tabla C.36: Prueba unitaria Clustering 8 - Validación de métodos de acceso (getter)

Prueba	UT-CLUS-009
Título	Validar consistencia en clustering con estimación automática
Precondiciones	Datos fijos
Cobertura	Método aplicar() - consistencia en 20 ejecuciones
Dependencias	scikit-learn
Pasos	1. Estimar k una vez, 2. Ejecutar clustering 20 veces
Configuración	Entrada: Datos: (15, 2) (3 clusters muy separados)
Esperado	20 ejecuciones idénticas (KMeans con k=3 automáticos)
Resultado	20 ejecuciones consistentes (clúster con k=3)
Estado	**APROBADO**

Tabla C.37: Prueba unitaria Clustering 9 - Consistencia con estimación automática de k

Prueba	UT-CLUS-010
Título	Validar clustering con datos ruidosos y estimación automática
Precondiciones	Datos con ruido añadido
Cobertura	Método aplicar() - robustez a ruido
Dependencias	scikit-learn
Pasos	1. Añadir ruido a datos, 2. Estimar clusters, 3. Aplicar clustering
Configuración	Entrada: Datos: (30, 5) (con ruido significativo)
Esperado	Clustering robusto a datos ruidosos con estimación automática
Resultado	Clustering exitoso. k estimado: 2 (Silhouette: 0.487). 2 clusters encontrados.
Estado	**APROBADO**

Tabla C.38: Prueba unitaria Clustering 10 - Robustez a datos ruidosos

Resumen de ejecución: Ran 10 tests in 9.26s
Estadísticas finales:

- **Total de pruebas:** 10
- **Pruebas aprobadas:** 10 (100 %)
- **Tiempo total de ejecución:** 9.26 segundos

Reporte de Pruebas Unitarias: Procesador Clasificación

Prueba	UT-CLAS-001
Título	Validar inicialización del clasificador
Precondiciones	Ninguna
Cobertura	Constructor __init__()
Dependencias	Ninguna
Pasos	1. Crear instancia de ProcesadorClasificacion
Esperado	Clasificador inicializado, SVM no cargado aún
Resultado	SVM cargado: False, Modelo: NO
Estado	**APROBADO**

Tabla C.39: Prueba unitaria Clasificación 1 - Inicialización

Prueba	UT-CLAS-002
Título	Validar carga de modelo sin excepción
Precondiciones	Ninguna
Cobertura	Método cargar_modelo_svm() - manejo de errores
Dependencias	ProcesadorSVMClasificador real
Pasos	1. Llamar cargar_modelo_svm() y verificar que no crashea
Configuración	Modelo LinearSVC cargado con Clases disponibles: ['derecha', 'funciona.', 'izquierda', 'no_politico']
Esperado	Método ejecutado sin excepción crítica
Resultado	Modelo SVM cargado exitosamente. Clases disponibles: ['derecha', 'funciona.', 'izquierda', 'no_politico']. Método ejecutado, retornó tipo: ProcesadorSVMClasificador
Estado	**APROBADO**

Tabla C.40: Prueba unitaria Clasificación 2 - Carga de modelo

Prueba	UT-CLAS-003
Título	Validar palabras representativas
Precondiciones	Mapeo palabra-vector disponible
Cobertura	Método obtener_palabras_representativas()
Dependencias	numpy
Pasos	1. Calcular palabras más cercanas al centroide
Configuración	Frases en cluster: ['frase1', 'frase2', 'frase3']
Esperado	Lista de 2 frases más cercanas al centroide
Resultado	2 frases: ['frase2', 'frase1']
Estado	**APROBADO**

Tabla C.41: Prueba unitaria Clasificación 3 - Palabras representativas

Prueba	UT-CLAS-004
Título	Validar palabras representativas sin vectores
Precondiciones	Frases sin vectores en mapeo
Cobertura	Método obtener_palabras_representativas() - caso vacío
Dependencias	numpy
Pasos	1. Pasar frases no existentes en mapeo
Configuración	Frases sin vectores: ['fraseX', 'fraseY', 'fraseZ']
Esperado	Lista vacía (ninguna frase tiene vector)
Resultado	Lista con 0 elementos: []
Estado	**APROBADO**

Tabla C.42: Prueba unitaria Clasificación 4 - Caso sin vectores

Prueba	UT-CLAS-005
Título	Validar clasificación no político (90 %+)
Precondiciones	Predicciones mayoritariamente no político
Cobertura	Método determinar_ideologia_final() - umbral 90 %
Dependencias	Ninguna
Pasos	1. Proporcionar 9 no_politico y 1 político
Configuración	Frases: 10 total (9 no_politico, 1 izquierda)
Esperado	no_politico (90 % no político)
Resultado	no_politico
Estado	**APROBADO**

Tabla C.43: Prueba unitaria Clasificación 5 - Umbral alto no político

Prueba	UT-CLAS-006
Título	Validar clasificación con mayoría clara
Precondiciones	7 izquierda, 3 derecha
Cobertura	Método determinar_ideologia_final() - umbral 60 %
Dependencias	Ninguna
Pasos	1. Proporcionar 7 izquierda, 3 derecha
Configuración	Frases: 10 total (7 izquierda, 3 derecha)
Esperado	izquierda (70 % >60 %)
Resultado	izquierda
Estado	**APROBADO**

Tabla C.44: Prueba unitaria Clasificación 6 - Mayoría clara

Prueba	UT-CLAS-007
Título	Validar clasificación con empate
Precondiciones	5 izquierda, 5 derecha
Cobertura	Método determinar_ideologia_final() - empate
Dependencias	Ninguna
Pasos	1. Proporcionar empate 5-5
Configuración	Frases: 10 total (5 izquierda, 5 derecha)
Esperado	neutral (empate 5-5)
Resultado	neutral
Estado	**APROBADO**

Tabla C.45: Prueba unitaria Clasificación 7 - Caso empate

Prueba	UT-CLAS-008
Título	Validar clasificación mayoría débil + económico
Precondiciones	6 izquierda, 4 derecha, tema económico
Cobertura	Método determinar_ideologia_final() - mayoría débil
Dependencias	Ninguna
Pasos	1. Proporcionar 6-4 con tema económico
Configuración	Frases: 10 total (6 izquierda, 4 derecha) Tema económico: SÍ
Esperado	izquierda (60 %, tema económico)
Resultado	izquierda
Estado	**APROBADO**

Tabla C.46: Prueba unitaria Clasificación 8 - Mayoría débil con tema económico

Prueba	UT-CLAS-009
Título	Validar poco político sin económico
Precondiciones	2 políticas, 8 no políticas, sin tema económico
Cobertura	Método determinar_ideologia_final() - mínimo 2 políticas
Dependencias	Ninguna
Pasos	1. Proporcionar 2 políticas, 8 no políticas, sin económico
Configuración	Frases: 10 total (2 políticas, 8 no políticas) Tema económico: NO
Esperado	no_politico (solo 20 % político, sin tema económico)
Resultado	neutral
Estado	**FALLIDO**

Tabla C.47: Prueba unitaria Clasificación 9 - Poco contenido político

Prueba	UT-CLAS-010
Título	Validar consistencia en clasificación
Precondiciones	Mismos datos de entrada
Cobertura	Método determinar_ideologia_final() - 20 ejecuciones
Dependencias	Ninguna
Pasos	1. Ejecutar 20 veces con mismos datos
Configuración	Datos de prueba consistentes para múltiples ejecuciones
Esperado	20 resultados idénticos (algoritmo determinístico)
Resultado	20 ejecuciones con 1 único(s) - {'izquierda'}
Estado	**APROBADO**

Tabla C.48: Prueba unitaria Clasificación 10 - Consistencia

Resumen de ejecución: Ran 10 tests in 0.028s
Estadísticas finales:

- **Total de pruebas:** 10
- **Pruebas aprobadas:** 9 (90 %)
- **Pruebas fallidas:** 1 (10 %) - UT-CLAS-009
- **Tiempo total de ejecución:** 0.028 segundos

Reporte de Pruebas Unitarias: Procesador PLDA

Prueba	UT-PLDA-001
Título	Validar inicialización del procesador
Precondiciones	Ninguna
Cobertura	Constructor __init__() - diccionarios cargados
Dependencias	Diccionario MARPOR
Pasos	1. Crear instancia de ProcesadorEconomicoPLDA
Esperado	Procesador inicializado con diccionarios MARPOR
Resultado	Diccionario: True, Mapeo: True, Nombres: True
Estado	**APROBADO**

Tabla C.49: Prueba unitaria PLDA 1 - Inicialización

Prueba	UT-PLDA-002
Título	Validar detección de tema económico
Precondiciones	Frases con contenido económico claro
Cobertura	Método identificar_tema_plda() - detección positiva
Dependencias	Diccionario MARPOR
Pasos	1. Procesar frases con contenido económico
Configuración	Frases: 3 con contenido económico
Esperado	Tema económico detectado (es_economico=True)
Resultado	es_economico: True, tema: Dimensión Económica - Izquierda
Estado	**APROBADO**

Tabla C.50: Prueba unitaria PLDA 2 - Detección económica positiva

Prueba	UT-PLDA-003
Título	Validar no detección con frases no económicas
Precondiciones	Frases sin contenido económico
Cobertura	Método identificar_tema_plda() - detección negativa
Dependencias	Diccionario MARPOR
Pasos	1. Procesar frases sin contenido económico
Configuración	Frases: 4 sin contenido económico
Esperado	No tema económico (es_economico=False)
Resultado	es_economico: False, tema: No económico
Estado	**APROBADO**

Tabla C.51: Prueba unitaria PLDA 3 - Detección negativa

Prueba	UT-PLDA-004
Título	Validar manejo de lista vacía
Precondiciones	Lista vacía de frases
Cobertura	Método identificar_tema_plda() - entrada vacía
Dependencias	Diccionario MARPOR
Pasos	1. Procesar lista vacía
Configuración	Frases: lista vacía
Esperado	No económico con lista vacía
Resultado	es_economico: False, tema: No económico
Estado	**APROBADO**

Tabla C.52: Prueba unitaria PLDA 4 - Entrada vacía

Prueba	UT-PLDA-005
Título	Validar estructura completa del resultado
Precondiciones	Frases económicas
Cobertura	Método identificar_tema_plda() - estructura de salida
Dependencias	Diccionario MARPOR
Pasos	1. Verificar todas las claves del diccionario resultado
Configuración	Entrada: intancia de idetificar_Tema_plda()
Esperado	Diccionario con 10 claves
Resultado	10 claves: ['tema', 'orientacion', 'orientacion_palabras_clave', 'palabras_clave', 'palabras_economicas', 'frases_asociadas', 'patrones_detectados', 'score_total', 'es_economico', 'metodo']
Estado	**APROBADO**

Tabla C.53: Prueba unitaria PLDA 5 - Estructura de resultado

Prueba	UT-PLDA-006
Título	Validar detección orientación izquierda
Precondiciones	Frases con contenido de izquierda
Cobertura	Método identificar_tema_plda() - orientación izquierda
Dependencias	Diccionario MARPOR
Pasos	1. Procesar frases de regulación/nacionalización
Configuración	Frases: 3 con temática de izquierda (“El gobierno debe nacionalizar...", "Se necesita mayor regulación...", "La planificación económica...")
Esperado	Orientación izquierda detectada
Resultado	Orientación: izquierda
Estado	**APROBADO**

Tabla C.54: Prueba unitaria PLDA 6 - Orientación izquierda

Prueba	UT-PLDA-007
Título	Validar detección orientación derecha
Precondiciones	Frases con contenido de derecha
Cobertura	Método identificar_tema_plda() - orientación derecha
Dependencias	Diccionario MARPOR
Pasos	1. Procesar frases de libre mercado/incentivos
Configuración	Frases: 3 con temática de derecha (“El libre mercado es la solución para...", "Se necesitan incentivos fiscales...", "La desregulación económica...")
Esperado	Orientación derecha detectada
Resultado	Orientación: derecha
Estado	**APROBADO**

Tabla C.55: Prueba unitaria PLDA 7 - Orientación derecha

Prueba	UT-PLDA-008
Título	Validar palabras clave no vacías
Precondiciones	Tema económico detectado
Cobertura	Método identificar_tema_plda() - palabras clave
Dependencias	Diccionario MARPOR
Pasos	1. Verificar que palabras_clave no esté vacío
Configuración	Frases: 2 económicas (“El gobierno debe regular...", "La nacionalización de la industria...")
Esperado	Lista de palabras clave no vacía
Resultado	2 palabras clave
Estado	**APROBADO**

Tabla C.56: Prueba unitaria PLDA 8 - Palabras clave

Prueba	UT-PLDA-009
Título	Validar consistencia en detección
Precondiciones	Mismas frases de entrada
Cobertura	Método identificar_tema_plda() - 20 ejecuciones
Dependencias	Diccionario MARPOR, tomotopy
Pasos	1. Ejecutar 20 veces con mismas frases económicas
Configuración	Entrada: 2 frases económicas fijas (“El gobierno debe aumentar el gasto público", “Se necesita más regulación estatal")
Esperado	20 resultados consistentes (mismo es_economico)
Resultado	Ejecutando 20 veces... Resultados: 20/20 positivos, consistentes: True
Estado	**APROBADO**

Tabla C.57: Prueba unitaria PLDA 9 - Consistencia en detección

Prueba	UT-PLDA-010
Título	Validar mapeo de orientación
Precondiciones	Mapeo de categorías a orientaciones
Cobertura	Atributo mapeo_orientacion - coherencia
Dependencias	Ninguna
Pasos	1. Verificar que todas las categorías en mapeo existen en diccionario
Configuración	Mapeo interno del procesador PLDA
Esperado	Todas las categorías del mapeo existen en diccionario
Resultado	Categorías mapeo: 9, Faltantes: 0 Orientaciones usadas: {`'derecha'`,`'izquierda'`} Orientaciones válidas: {`'derecha'`,`'izquierda'`}
Estado	**APROBADO**

Tabla C.58: Prueba unitaria PLDA 10 - Mapeo de orientación

Resumen de ejecución: Ran 10 tests in 0.039s
Estadísticas finales:

- **Total de pruebas:** 10
- **Pruebas aprobadas:** 10 (100 %)
- **Pruebas fallidas:** 0 (0 %)
- **Tiempo total de ejecución:** 0.039 segundos

Reportes de Pruebas de Integración

Las pruebas de integración verifican la interacción correcta entre los diferentes módulos del sistema.

Reporte de Prueba de Integración: Procesador NLP - Procesador Embeddings

Prueba	IT-PIPE-001
Título	Validar integración NLP → Embeddings
Precondiciones	Módulos NLP y Embeddings disponibles
Cobertura	Transformación texto → embeddings
Dependencias	spaCy, SentenceTransformer
Entrada	3 artículos simulados: izquierda, derecha, no político
Proceso	Extracción frases + generación embeddings (768D)
Esperado	3 embeddings de 768 dimensiones
Resultado	3 embeddings generados, dimensión (768,) cada uno
Estado	**APROBADO**

Tabla C.59: Prueba de integración NLP → Embeddings (IT-PIPE-001)

Resumen de ejecución: PASSED

- **Frases procesadas:** 3 artículos → 3 frases extraídas
- **Embeddings generados:** 3 embeddings de 768 dimensiones
- **Flujo verificado:** Texto → NLP → Embeddings → Vectores
- **Estado:** APROBADO

Reporte de Prueba de Integración: Procesador Embeddings - Procesador Autoencoder

PRUEBA DE INTEGRACIÓN: Embeddings → Autoencoder IT-PIPE-002: Validar reducción dimensional

Prueba	IT-PIPE-002
Título	Validar integración Embeddings → Autoencoder
Precondiciones	Embeddings 768D disponibles
Cobertura	Reducción dimensional 768D → 4D
Dependencias	TensorFlow/Keras
Entrada	Embeddings 768D, configuración: 768→4→768
Proceso	Crear y configurar autoencoder
Esperado	Autoencoder configurado: input=768, latent=4
Resultado	Configuración correcta, reducción: 0.52 %
Estado	**APROBADO**

Tabla C.60: Prueba de integración Embeddings → Autoencoder (IT-PIPE-002)

Resumen de ejecución: PASSED

- **Tiempo de ejecución:** 40.29 segundos (incluye carga de TensorFlow)
- **Integración verificada:** Configuración correcta del autoencoder para recibir embeddings
- **Dimensiones validadas:** 768D → 4D confirmadas
- **Estado:** APROBADO

Reporte de Prueba de Integración: Procesador Autoencoder - Procesador Clustering

Prueba	IT-PIPE-003
Título	Validar integración Autoencoder → Clustering
Precondiciones	Vectores latentes 4D generados
Cobertura	Agrupación en espacio latente
Dependencias	scikit-learn
Entrada	20 vectores 4D, estimación automática k=2-5
Proceso	Estimar k óptimo + aplicar clustering
Esperado	Clusters formados con k estimado
Resultado	4 clusters encontrados, distribución: {3,8,5,4}
Estado	**APROBADO**

Tabla C.61: Prueba de integración Autoencoder → Clustering (IT-PIPE-003)

Resumen de ejecución: PASSED

- **Tiempo de ejecución:** 45.22 segundos (incluye estimación y clustering)
- **Integración verificada:** Flujo Autoencoder → Clustering funciona correctamente
- **Estimación automática:** El sistema estima correctamente 4 clusters
- **Estado:** APROBADO

Reporte de Prueba de Integración: Procesador Clustering - Procesador Clasificación

Prueba	IT-PIPE-004
Título	Validar integración Clustering → Clasificación → PLDA
Precondiciones	Clusters formados, datos de frases
Cobertura	Interpretación clusters + clasificación + PLDA
Dependencias	Clasificación ideológica, procesador PLDA
Entrada	12 frases en 3 clusters, mezcla económica
Proceso	Interpretar clusters + clasificar + análisis PLDA
Esperado	3 clusters interpretados con PLDA integrado
Resultado	Cluster 0: Derecha (econ), Cluster 1: No político, Cluster 2: Izquierda (econ)
Estado	**APROBADO**

Tabla C.62: Prueba de integración Clustering → Clasificación → PLDA (IT-PIPE-004)

Resumen de ejecución: PASSED

- **Tiempo de ejecución:** 31.34 segundos (procesamiento completo)
- **Integración verificada:** Flujo Clustering → Clasificación → PLDA funciona correctamente
- **PLDA integrado:** Campo 'analisis_económico' presente en 100 % de los clusters
- **Clasificación ideológica:** Funcionando (Derecha, no_politico, Izquierda)

- **Análisis económico:** 2/3 clusters con contenido económico detectado
- **Estado:** APROBADO

Reporte de Prueba de Integración: Pipeline Completo

Prueba	IT-PIPE-REAL-005
Título	Validar pipeline completo con datos de prueba
Precondiciones	Todos los módulos disponibles
Cobertura	Ejecución secuencial completa (5 etapas)
Dependencias	Todos los módulos del sistema
Entrada	3 artículos: izquierda, derecha, no político
Proceso	NLP $\rightarrow$ Embeddings $\rightarrow$ Autoencoder $\rightarrow$ Clustering $\rightarrow$ Clasificación
Esperado	Flujo completo sin interrupciones
Resultado	Todas las etapas completadas exitosamente
Estado	**APROBADO**

Tabla C.63: Prueba de integración Pipeline Completo (End-to-End)

Resumen de ejecución: PASSED

- **Tiempo de ejecución:** Completo (registro detallado por etapas)
- **Integración verificada:** Pipeline completo funcionando correctamente
- **Etapas completadas:** 5/5 etapas del flujo NLPolitik
- **PLDA integrado:** Confirmado en etapa de clasificación
- **Estado:** APROBADO